



# Guia de integração — PWA + Anota AI

Como colocar o PWA da Gringo Pizzaria no ar substituindo o cardápio digital que a Anota AI fornece ao cliente final, mantendo todo o resto do sistema Anota AI funcionando como está: robô de atendimento no WhatsApp, painel de pedidos, cozinha, impressão e entrega.

Importante: o PWA não processa pagamentos. O cliente final apenas informa como pretende pagar (Pix, cartão ou dinheiro) e o acerto acontece na entrega/retirada, exatamente como já acontece hoje.

## Como a integração funciona



- O cliente acessa o PWA (ou instala na tela inicial), monta o pedido e confirma.
- O app abre o WhatsApp do próprio cliente com o pedido completo já formatado (itens, observações, endereço, forma de pagamento e total), endereçado ao número da pizzaria — o mesmo atendido pelo robô da Anota AI.
- O pedido chega como mensagem no atendimento. A equipe confirma e lança no painel da Anota AI, e o fluxo segue normal: cozinha, impressão, entrega.

O que muda: apenas a "vitrine" (o link de cardápio que o cliente final abre). O que não muda: nada na operação da Anota AI.

## Pré-requisitos

- Acesso ao painel administrativo da Anota AI do cliente.
- Número de WhatsApp da pizzaria (o atendido pelo robô da Anota AI).
- Hospedagem estática com HTTPS (GitHub Pages, Vercel, Netlify ou Cloudflare Pages — o plano gratuito atende).
- Opcional: domínio próprio (ex.: pedido.gringopizzaria.com.br).



## Passo 1 — Configurar o app (config.js)

Edite o arquivo `config.js` na raiz do projeto:

```
window.GRINGO_CONFIG = {  
  // OBRIGATÓRIO: 55 + DDD + número, só dígitos.  
  // É para este número que os pedidos serão enviados.  
  whatsapp: "5511999999999",  
  
  // Opcional (deixe "" por enquanto): URL de webhook para a  
  // integração direta via API – ver "Upgrade opcional" no fim.  
  webhookPedidos: "",  
  
  // Taxa de entrega em reais.  
  taxaEntrega: 7,  
};
```

Sem o whatsapp preenchido o pedido não é enviado a lugar nenhum — este é o único passo obrigatório no código.

## Passo 2 — Conferir o cardápio e os dados da loja

Tudo que o cliente final vê está em `index.html`, nas constantes no início do script da aplicação. Confira com o dono da pizzaria antes de publicar:

| O que conferir                                            | Onde está                                              |
|-----------------------------------------------------------|--------------------------------------------------------|
| Sabores, ingredientes e preços (por linha e tamanho)      | constante SABORES                                      |
| Linhas (Sem Borda / Com Borda / Premium) e recheios       | LINHAS, RECHEIOS, PREMIOS                              |
| Taxa de entrega                                           | <code>config.js</code> ( <code>taxaEntrega</code> )    |
| Desconto de 10% no Pix (regra ativa no app!)              | <code>busque subtotal * 0.1</code>                     |
| Horário (hoje: qua-seg, 18h-23h, fechado terça)           | <code>busque aberto</code> e <code>horarioTexto</code> |
| Endereço exibido (hoje: Rua Professor Sud Menucci, 499-A) | <code>busque localTexto</code>                         |
| Fotos das pizzas                                          | <code>pasta assets/</code>                             |

## Passo 3 — Publicar o PWA

O projeto é 100% estático: não tem build nem backend. Basta servir a pasta.

### Opção A — GitHub Pages (grátis, direto do repositório)

- Faça o merge do pull request para a branch `main`.
- No GitHub: Settings → Pages → Deploy from a branch → `main` / (root).
- A URL será `https://<usuario>.github.io/GringoPizzaria-Front/`.

### Opção B — Vercel / Netlify / Cloudflare Pages

- Importe o repositório no serviço.



- Framework: nenhum; comando de build: nenhum; pasta de saída: raiz (.).
- Opcional: aponte um domínio próprio (ex.: pedido.gringopizzaria.com.br).

HTTPS é obrigatório — sem ele o PWA não instala e o service worker não funciona (as hospedagens acima já entregam HTTPS sozinhas). Após qualquer atualização publicada, aumente `CACHE_VERSION` no `sw.js` (ex.: v1 → v2) para que os celulares dos clientes baixem a versão nova.

## Passo 4 — Substituir a página da Anota AI pelo PWA

Este é o passo que efetivamente "troca a vitrine". O link do PWA precisa entrar em todos os lugares onde hoje o cliente recebe o link do cardápio da Anota AI:

- Robô da Anota AI (principal): no painel, edite as mensagens automáticas do robô (saudação / envio de cardápio) para enviarem o link do PWA no lugar do link `pedido.anota.ai`. O caminho exato varia conforme o plano, mas fica nas configurações de atendimento/mensagens do robô. Se a opção não aparecer, peça ao suporte da Anota AI para personalizar a mensagem de cardápio.
- Bio do Instagram da pizzaria.
- Google Meu Negócio (botão "Fazer pedido" / site).
- QR codes de mesa, balcão, panfletos e embalagens — gere novos apontando para o PWA.
- Catálogo/recados do WhatsApp Business, se houver link fixado.

## Passo 5 — Ajustar o robô para receber os pedidos

O pedido chega ao WhatsApp como uma mensagem começando com **\*NOVO PEDIDO — Gringo Pizzaria\***. Configure o comportamento do robô para esse caso:

- O ideal é o robô transferir para atendimento humano ao receber mensagem nesse formato (em vez de tentar interpretá-la como conversa). Isso se configura nas regras de transbordo/atendimento humano do robô.
- Alinhe com a equipe: ao receber o pedido, confirmar com o cliente e lançar no painel da Anota AI como já fazem com pedidos de WhatsApp. A partir daí cozinha e impressão seguem o fluxo normal.

## Passo 6 — Teste de ponta a ponta (checklist de go-live)

Faça um pedido real de teste, do celular, antes de divulgar:

- O PWA abre pela URL publicada e oferece "Adicionar à tela inicial".
- Instalado, abre em tela cheia com o ícone da pizzaria.
- Montar uma pizza (2 sabores, borda, observação) e finalizar o pedido.
- O WhatsApp abriu com a mensagem completa e endereçada ao número certo.
- O robô da Anota AI recebeu e transferiu para humano (não "conversou" com o pedido).
- A equipe lançou o pedido no painel e ele saiu na cozinha/impressora.
- Repetir o teste na opção Retirada e com cada forma de pagamento.
- Modo avião depois do primeiro acesso: o cardápio ainda abre (offline OK).



## Operação diária (resumo para a equipe)

- Chegou mensagem NOVO PEDIDO no WhatsApp → confirmar com o cliente.
- Lançar o pedido no painel da Anota AI (como qualquer pedido de WhatsApp).
- Cozinha, impressão e entrega: tudo igual a hoje.
- Pagamento: na entrega/retirada, conforme o cliente informou. O app não cobra nada.

## Limitações conhecidas

- O acompanhamento do pedido no app ("recebido → preparando → saiu → entregue") é uma estimativa local por tempo, não o status real do painel da Anota AI. Status real exige a integração via API (abaixo).
- O envio depende de o cliente concluir o envio da mensagem que o app abre no WhatsApp dele. Se ele fechar sem enviar, o pedido não chega (o app mantém o pedido na tela "Pedidos" do cliente, mas a pizzaria não recebe).

## Upgrade opcional — integração direta via API (sem WhatsApp)

Para o pedido cair direto no painel da Anota AI (e permitir status real do pedido no app):

- O cliente solicita ao suporte da Anota AI o acesso à API de parceiros (token de integração).
- Cria-se um pequeno intermediário que recebe o pedido do app e chama a API da Anota AI. A forma mais simples é um fluxo n8n com um nó Webhook — sem servidor próprio e sem repositório novo. Alternativa: uma função serverless na própria Vercel/Netlify onde o PWA estiver hospedado.
- Preencha `webhookPedidos` no `config.js` com a URL desse webhook. O app passa a enviar o pedido completo em JSON via POST a cada confirmação — nenhuma alteração de código é necessária.

O envio por WhatsApp e o webhook funcionam em paralelo: dá para ativar o webhook e manter o WhatsApp como redundância durante a transição.

## Referência rápida de arquivos

| Arquivo                                                           | Para que serve                                                                  |
|-------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <code>config.js</code>                                            | WhatsApp, webhook opcional e taxa de entrega — único arquivo que precisa editar |
| <code>index.html</code>                                           | App completo (cardápio, preços, montador, carrinho, checkout, acompanhamento)   |
| <code>sw.js</code>                                                | Service worker — suba <code>CACHE_VERSION</code> a cada deploy                  |
| <code>manifest.webmanifest</code> + <code>icons/</code>           | Identidade do app instalado                                                     |
| <code>assets/</code> , <code>fonts/</code> , <code>vendor/</code> | Fotos, vídeo, fontes e React locais (sem CDN)                                   |

Este documento acompanha o repositório GringoPizzaria-Front (arquivo `INTEGRACAO-ANOTA-AI.md`). Em caso de divergência, o arquivo do repositório é a versão mais atual.